

i want to make a software that can work like NWS breach or BRCH GUI or FLDWAV. these software help in analysis of a dam breach. it produces necessary outflow hydrograph, breach time, breach dimensions etc. all these data are necessary to make a hydrodynamic modelling. do not produce anything yet. i want to talk about this idea first. what does hecras do in this case. how it works does it produce these data or it ask for data to produce hydrodynamic modelling. when i was studying the Glacial lake. there were no softwares that could provide the outflow hydrograph and other data. those that did had very difficult gui and we spent like 2 months just to know how to use those software. so what do you think ?

Thought for 21s

Good, let's talk it through before touching any code.

What HEC-RAS actually does

HEC-RAS isn't a breach-prediction tool at its core — it's a hydraulic/hydrodynamic routing engine. For dam breach analysis it does two things bolted together:

1. **Breach definition (input, mostly).** You tell it the breach geometry (bottom width, side slopes, invert elevation) and the time to full formation, either by typing these numbers in yourself or by picking one of the built-in parametric regression equations (Froehlich 1995/2016, MacDonald & Langridge-Monopolis, Von Thun & Gillette). Since HEC-RAS 5.x it also has an optional physically-based breach progression model (erosion-based, similar in spirit to what BREACH does) where you give it soil/erodibility parameters and it computes breach growth over time instead of you assuming a triangle/trapezoid growing linearly. But even this "physical" mode is really just one more sub-model plugged into the same interface — it's not the heart of the software.
2. **Routing (the actual core).** Once breach geometry vs. time exists, HEC-RAS computes the outflow hydrograph itself via reservoir mass-balance + weir/orifice hydraulics through the breach, then routes that hydrograph downstream using 1D (or 2D) unsteady flow — full Saint-Venant equations, same as FLDWAV does.

So HEC-RAS is really "BREACH's breach module (optional, simplified) + FLDWAV's routing engine + a modern GUI," all under one roof. That consolidation is exactly why it displaced the older USACE/NWS tools for most standard dam types — you're not stitching together three different programs and reformatting fixed-column text files between them.

Where the older tools differ

- **NWS BREACH** is narrowly focused: given dam material properties (grain size, cohesion, etc.) it simulates the physical erosion process and *outputs* the breach hydrograph,

geometry, and time — you don't prescribe those, the model derives them from soil mechanics. That's its real value over HEC-RAS's simplified/parametric options.

- **FLDWAV** only does downstream unsteady routing — it takes a breach hydrograph as an upstream boundary condition (often produced by BREACH or DAMBRK) and propagates it through the channel/floodplain.
- **BRCH GUI** is just a wrapper trying to make BREACH usable without hand-editing input decks.

Where this leaves glacial lake outburst floods (GLOF)

This is the real gap, and I think you're right that it's underserved. The tools above were built around engineered embankment/concrete dams — soil properties, controlled construction, known material strength. A moraine dam breach in a GLOF context is a different animal:

- Breach initiation is usually overtopping or piping through poorly sorted, ice-cored, highly heterogeneous moraine material — the soil-mechanics assumptions in BREACH don't transfer cleanly.
- People studying GLOFs often fall back on empirical peak-discharge regressions (Costa, Walder & O'Connor, Froehlich) calibrated on a mix of moraine and embankment failures, which is a much cruder approximation than a physically-based erosion model would give you.
- There isn't a well-adopted, purpose-built, physically-based moraine-breach model with a decent interface — which matches your experience.

On the idea itself

I think there's a legitimate niche here, but it's worth being precise about which problem you're solving, because "BREACH/FLDWAV but with a better GUI" and "a moraine-dam-specific breach model" are different projects:

- If the pain point is *usability* of existing, scientifically-accepted methods (Froehlich regressions, NWS BREACH's erosion approach, even HEC-RAS's own breach module) — you could build a modern, opinionated front-end that walks a user through the inputs, validates them, and calls into an open, well-documented implementation of these established equations/models, then hands off a clean hydrograph for hydrodynamic routing (HEC-RAS 2D, or your own).
- If the pain point is *scientific applicability to moraine dams* — that's a more ambitious, research-grade project: you'd be implementing or adapting an erosion model for non-

cohesive, ice-cored material, which is its own research contribution, not just a GUI problem.

Given you mention two months just learning the interface of existing tools, my guess is a lot of the value you'd add is (a) sane defaults and guided input for people who know hydrology but not the software's quirks, and (b) transparent documentation of which equation/assumption is being used where, since these tools are notorious for being black boxes about that.

What's your actual target: general dam breach analysis (any dam type), or specifically GLOF/moraine dams? That changes a lot about which breach model(s) you'd want to build around.

i think we can do both. it would make the software more usable for a wide range of work. i want to make the software as graphical as possible so that students with a little knowledge can use it too. so what would be the initial steps how can we do it ?

Good — building for both general dam breach and GLOF/moraine cases from day one is doable if you architect it as one routing/output core with swappable breach modules, rather than two separate programs. Here's how I'd think about the initial steps.

1. Nail down scope before writing anything

Decide the boundary of your software honestly:

- Are you doing breach parameter generation only (hydrograph, breach geometry/time) and letting the user export to HEC-RAS/Iber/TELEMAC for downstream routing?
- Or are you also doing the downstream hydrodynamic routing yourself (1D/2D unsteady flow)?

These are very different amounts of work. The routing engine (Saint-Venant solver, mesh generation, boundary conditions) is a multi-year undertaking on its own — it's what makes HEC-RAS, TELEMAC, etc. serious pieces of engineering. My honest suggestion: **for v1, focus entirely on the breach module** — outflow hydrograph, breach geometry evolution, breach time — and export a clean hydrograph (time vs. discharge) that plugs into existing, trusted routing tools (HEC-RAS 2D being the obvious target since it's free and dominant). This alone solves the actual pain point you described from your glacial lake work. Routing can be phase 2, or you can integrate HEC-RAS as a backend later via its computational libraries rather than reinventing Saint-Venant.

2. Design the breach module as a set of interchangeable models

Structure it so a user picks (or you auto-suggest based on dam type) one of:

- Parametric/regression models: Froehlich (1995, 2016), MacDonald & Langridge-Monopolis, Von Thun & Gillette — these are just equations, easy to implement, good for "I have limited data" cases.
- Physically-based erosion model: NWS BREACH-style, for cohesive/engineered embankments.
- A moraine/GLOF-specific erosion model — this is your novel contribution. You'd likely need to build or adapt one from literature (Walder & O'Connor, Westoby et al., Kirschbaum-type moraine mechanics papers) since nothing standard exists.

Each model has its own input set. Your GUI's job is to only show the inputs relevant to the selected model, with sane defaults and inline explanations — that directly solves the "2 months to learn the interface" problem.

3. Decide your core data flow

Regardless of model chosen, standardize on one output contract: time series of breach width/depth, and time series of outflow discharge, in a clean format (CSV/JSON) so it's trivial to hand off to HEC-RAS, FLDWAV, or your own future router. This lets you build the GUI and the science somewhat independently.

4. Pick a tech stack suited to "graphical and easy"

For something students can pick up fast:

- Desktop app (Python + Qt/PySide, or Electron) if you want offline use and easy distribution.
- Or a web app (Python/FastAPI backend + React frontend) if you want zero-install and easier iteration/updates — probably better for student adoption since there's nothing to install.
- Either way, breach hydraulics/erosion equations are straightforward numerically (Python + NumPy/SciPy is more than enough — no need for compiled code unless you're doing your own 2D routing later).

5. UI/UX approach for a non-expert audience

- Wizard-style flow: dam type → data availability → guided input → results, rather than a dense all-at-once form (this is the single biggest thing wrong with BREACH/BRCH GUI/FLDWAV).
- Live plots of breach growth and hydrograph as parameters change, not just a final table.

- Built-in tooltips/citations for every equation used — addresses your "black box" complaint and builds trust for academic use.
- Sensible unit handling (metric default, since most GLOF work is outside the US).

6. Validate early against known cases

Pick 2–3 well-documented historical breaches (a classic embankment failure with good records, plus a documented GLOF like a Nepalese/Himalayan or Peruvian moraine breach) and make sure your outputs are in the right ballpark against published results. This becomes your credibility anchor for a scientific tool.

Suggested first milestone

A working single-page tool: user picks dam type + breach model, enters a handful of parameters, gets a hydrograph plot + breach geometry table + CSV export. No routing, no fancy features. That's genuinely useful on day one and testable with real users (maybe former classmates who did GLOF work) before you invest in the harder physically-based moraine model or any routing engine.

Want to go deeper into any one of these next — e.g., picking the actual breach equations/model to implement first, or sketching the wizard flow/UI screens?

Searched the web

Excellent — I now have a solid, well-verified basis for both parts of your question. Let me search one more thing on the moraine-specific breach mechanics before I put it all together.

Searched the web

Good — I now have solid backing for both pieces of this. Let me lay it out.

1. Estimating lake volume/depth when bathymetry is unavailable

This is a real, well-documented problem — not something specific to your case. The standard workaround is **volume–area (V-A) empirical scaling**, the same family your Barun paper used (Sakai 2012). Here are the main relationships in use, with their actual form and, importantly, their known reliability — because the reliability varies a lot and matters for what you build.

O'Connor et al. (2001) — moraine-dammed lakes, Oregon Cascades:

$$V = 3.114A + 0.0001685A^2$$

Huggel et al. (2002) — mixed ice-dammed/moraine-dammed/thermokarst, global:

$$D = 0.104A^{0.42} \quad V = 0.104A^{1.42}$$

(Cook & Quincey's 2015 re-derivation from the same underlying data gives slightly revised coefficients: $D = 0.1217A^{0.4129}$, $V = 0.1217A^{1.4129}$ — either version gives similar results in practice.)

Evans (1986) / Canadian Inland Water Directorate — ice-dammed lakes:

$$V = 0.035A^{1.5}$$

Sakai (2012) — the one your paper used, regionally calibrated for Nepal/Tibet/Bhutan:

$$V = 43.24 \times A^{1.530} \quad D_{max} = 95.665 \times A^{0.489}$$

Which one to trust — this is the important part. Cook & Quincey (2015) ran a systematic error analysis against 69 bathymetrically-surveyed lakes and found real, quantified problems worth building directly into your software:

- Lake area and depth are only **moderately correlated** ($r^2 = 0.38$ across their full compiled dataset) — for a given area, mean depth can vary by an **order of magnitude**.
- O'Connor's relationship "greatly overestimates" volume in most cases outside its original moraine-dam dataset — errors regularly exceeding –90%.
- Huggel and Evans perform noticeably better on average, but still swing ± 25 –400% depending on the specific lake.
- Crucially: **many Himalayan lakes specifically (including Lower Barun itself, in their table) are systematically underpredicted** by all three general relationships — meaning regionally-calibrated formulas like Sakai's are the better default for your use case, exactly as your paper did.
- They also identify *which lake types* are least predictable: lakes still developing supraglacially (shallow, sitting on stagnant ice), lakes in basins with complex overdeepened bathymetry, and lakes formed by a glacier advancing across a valley. If any of these apply to a given lake, flag the volume estimate as low-confidence rather than presenting it as a solid number.

What this means for your software: don't hardcode a single V-A formula. Instead:

- Let the user pick a region-appropriate relationship (Sakai for Himalaya, O'Connor for Cascades-type settings, Huggel/Evans as generic fallbacks), the same way you're already doing with breach equations
- Surface the uncertainty explicitly — since Cook & Quincey give quantified error bands, you could show a plausible min–max volume range alongside the point estimate, not just one number
- Flag lakes matching the "unpredictable" categories above (supraglacial, complex bathymetry, valley-blocking) so a student knows to treat the derived volume cautiously
- This is honestly a great fit for the same side-by-side comparison + uncertainty visualization pattern you already wanted for the breach equations — same UI pattern, different variable

2. What moraine dam breach modeling needs to account for (the ice/soil-mechanics question)

The literature is clear that moraine dams are a genuinely different failure regime from engineered earthfill dams, not just a smaller/scaled version. Westoby et al. (2014) — the standard review on this — lays out the structure clearly:

Triggers specific to moraine dams (beyond generic overtopping/piping):

- Contact glacier calving displacing lake water
- Icefall from hanging glaciers
- Rock/ice/snow avalanches (your Barun case)
- Dam settlement and/or piping
- **Ice-cored moraine degradation** — this is the one with no equivalent in engineered-dam breach literature
- Rapid input of water from supra-, en-, or subglacial sources
- Seismicity

Conditioning factors that make a given moraine dam more failure-prone: large lake volume, low width-to-height dam ratio, degradation of buried ice in the moraine structure, and limited freeboard.

Why the ice core changes the physics, specifically:

- A conventional earthfill dam's material properties (friction angle, cohesion, D50, unit weight — the parameters BREACH/BRCH need) are assumed roughly static over the

breach event. A moraine dam's aren't: as the buried ice core thaws, the dam **loses structural mass and strength over time independent of the breach event itself**, lowering the crest and reducing freeboard, which can trigger overtopping even without an external wave.

- One study I found gives concrete numbers on this: buried-ice thaw depth in a monitored moraine dam is currently around 3.4 m, projected to reach 9.5–22 m by end-of-century depending on emissions scenario — meaning the "dam material properties" input in your breach module isn't a fixed value for a moraine dam, it's potentially a slowly time-varying one, and climate-scenario dependent.
- The material itself is also fundamentally different from what Froehlich/MacDonald/Von Thun & Gillette were calibrated on: moraine material is **poorly sorted, unconsolidated glacial till** — a heterogeneous mix of boulders, gravel, and fines with no engineered compaction — not the relatively homogeneous, compacted earthfill those regressions assume. This is a big part of why the Cook & Quincey-style error analysis (and the standard breach equations generally) show Himalayan/moraine-dam cases as outliers.
- Breach initiation is often via **impulse-wave overtopping** (as in your Barun paper) rather than gradual seepage or steady overtopping — meaning the "trigger" input to your breach module often needs to come from an upstream avalanche/wave model (Heller & Hager-type, or the RAMMS + impact-wave approach your paper used) rather than a simple water-level threshold.

What HR-BREACH shows as the more rigorous alternative: the Lake Scientist summary references an HR-BREACH reconstruction of the 1985 Dig Tsho GLOF, tracking sediment evacuation rate through the breaching process directly — this is closer to what a moraine-appropriate physically-based model should look like: erosion driven by actual sediment transport of the till material, not an assumed trapezoidal growth curve borrowed from earthfill-dam studies.

What this means for your moraine breach module, concretely

- **Material input should be categorical and moraine-specific**, not the friction-angle/cohesion/D50 fields BREACH uses for engineered dams — think: ice-core presence (yes/no), estimated ice content (%), degree of consolidation (loose/moderate/dense till), freeboard, width-to-height ratio. These map more naturally to how a glaciologist would actually describe a moraine dam.
- **Keep the standard empirical equations available** (as your Barun paper did with Froehlich 1995) since they're the practical, low-data-requirement default most GLOF studies still use — but **label them clearly as calibrated on engineered/earthfill dams**,

not moraine dams, since that's a real, literature-backed limitation your users should see, not a footnote.

- **Add a separate trigger-assessment layer** upstream of the breach module itself — avalanche/rock-fall volume → impact velocity → impulse wave height/velocity → overtopping depth — since this is what actually initiates most Himalayan moraine breaches, and it's a genuinely different calculation from anything in the standard dam-breach literature.
- **Time-varying freeboard from ice-core thaw** could be a nice differentiating feature for scenario/climate-projection work — letting a user see how a shrinking freeboard changes overtopping susceptibility over a projected timeline, directly motivated by the thaw-depth data above.

Want to now sketch how these two new modules (lake volume estimator + moraine-specific breach/trigger module) fit into the overall software architecture we outlined earlier, or dig into the impulse-wave equations (Heller & Hager, or the model your paper cited — Evers et al. 2019) next since that's the piece connecting avalanche modeling to breach initiation?